

Cours 8 — Types construits : les tuples

Semaine 8 ■ Manuel Hachette ch. 4 (§ 4.1, 4.2) ■ 1 h

Objectifs

- Distinguer types simples et types construits
- Créer un tuple, accéder à ses éléments, le déconstruire
- Écrire des fonctions qui reçoivent et renvoient des tuples

1. Types simples, types construits

Depuis septembre nous manipulons des valeurs **simples** : un entier, un flottant, un booléen. Mais les données réelles vont par paquets : un point a deux coordonnées, un élève a un nom et trois notes, une image a des milliers de pixels...

Les **types construits** rassemblent plusieurs valeurs en une seule :

- le **tuple** (p-uplet) : groupe **figé** de valeurs — cette semaine ;
- le **tableau** (liste Python) : séquence **modifiable** — semaine 9 ;
- le **dictionnaire** : association clé → valeur — semaine 10.

2. Créer et lire un tuple

```
point = (3, 5)           # un couple
rgb = (255, 102, 0)      # un triplet (couleur orange)
eleve = ("Ada", 16, "1G2") # types mélangés autorisés
```

Accès par **indice**, qui commence à 0 :

```
point[0]    # 3
rgb[2]      # 0
len(rgb)    # 3
```

Un tuple est **immuable** : on ne peut pas le modifier après création. `point[0] = 7` provoque `TypeError`. Pour « changer » un tuple, on en crée un nouveau.

3. Déconstruire un tuple (affectation multiple)

```
x, y = point           # x vaut 3, y vaut 5
r, v, b = rgb           # trois affectations d'un coup
a, b = b, a             # l'échange de variables, enfin élégant !
```

C'est le mystère de la semaine 1 résolu : `a, b = b, a` construit le tuple `(b, a)` puis le déconstruit dans `a` et `b`.

4. Tuples et fonctions

Une fonction ne peut avoir qu'un **return**... mais il peut porter un tuple :

```
def divmod_maison(a, b):  
    """Renvoie (quotient, reste) de la division de a par b."""  
    return a // b, a % b  
  
q, r = divmod_maison(17, 5)    # q = 3, r = 2
```

On peut aussi parcourir un tuple, comme toute séquence :

```
for note in (12, 15, 9):  
    print(note)
```

5. Quand utiliser un tuple ?

Quand le paquet a une **structure fixe** et connue : un point (x, y) , une date (j, m, a) , une couleur (r, v, b) , un résultat multiple. Si la collection doit grandir ou changer, ce sera un tableau (semaine prochaine).

À retenir

- Tuple = groupe ordonné et **immuable** ; indices depuis 0 ; **len** donne la taille.
- Déconstruction : `x, y = point` ; échange : `a, b = b, a`.
- **return** `a, b` renvoie un tuple ; on le récupère par déconstruction.
- Structure fixe $\rightarrow$ tuple ; collection évolutive $\rightarrow$ tableau.